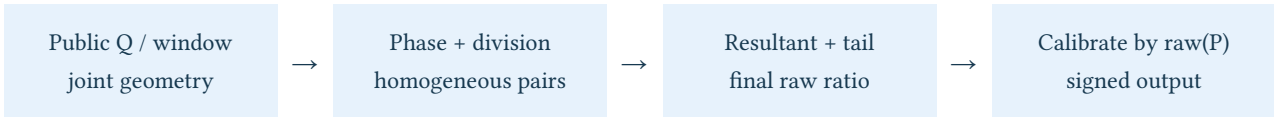
Native normalized source integration

Prime-field companion follow-up · 2026-11-03

A public-point API now joins the native phase/division, joint geometry and polynomial engines. It retains homogeneous recurrence pairs, resultant evaluation, partial blocks and singleton tails, then calibrates against raw(P). Identity maps to zero.



Checked: 3 saved raw records; 3 identity-crossing windows

Next: shared seed propagation, reusable strides, decoder integration

Checked evidence

Three targeted Rust methods pass in one invocation. Three historical setup records reproduce exact raw values, seeds, final ratios, resultants and partial polynomial values. Three length-three windows at O, P and minus P produce expected signs across identity. Scalar labels occur only in the checker. Sixteen dependency tests were filtered out.

The fixed checks imply 14 successful finite raw evaluations including calibration; this is a call-graph count, not instrumentation. Nine window outputs include two identities. Exact source/dependency hashes, compiler, commands and raw logs accompany receipt.json. Git commit completion lagged the tests; the frozen file hashes bind the tested bytes.

Scope and costs

Prime field and prime subgroup order are caller-certified. Phase enforces its torsion and coprime domain; inherited word bounds apply. This component recomputes phase per finite point, initializes a separate value engine and prepares strides per window call. Those costs are not yet shared. No full cohort, decoder recovery panel or timing comparison ran. Nonpartial raw branches need additional coverage.

Next construction

Preserve the original outer seed propagation and O/P/minus-P/2P anchor resets, retain prepared strides in reusable setup, then connect the repaired native decoder and count complete costs. The original 4563-case validation cohort is unchanged.

Sources

Historical shared-projective-grid/evaluator.py and normalized-partial-block/run.py, bound by original-inputs.sha256. See DERIVATION.md for formulas and the next seed recurrence. FLINT modular-polynomial documentation supplies context only; this report makes no external-backend performance claim.